

Broccoli: Brave OS-aware CC On Line

Design and Implementation of an OS-Integrated Minimal C Compiler
Targeting ARM64 (Apple Silicon)

Avin Mendiratta

Roll Number: 24124011

IDD, Mathematics and Computing

`avin.mendiratta.mat24@itbhu.ac.in`

February 18, 2026

1. Abstract

This project proposes the design and implementation of **Broccoli (Brave OS-aware CC On Line)**, a minimal yet non-trivial C subset compiler written in Kotlin that generates **ARM64 (AArch64) assembly** targeting Apple Silicon (macOS).

The primary objective is to connect compiler construction with Operating Systems concepts by generating real assembly that adheres to the ARM64 calling convention and produces valid Mach-O executables via the system toolchain.

Unlike conventional undergraduate compiler projects that terminate at AST or IR generation, Broccoli will:

- Generate ARM64 assembly compatible with Apple Silicon.
- Follow the macOS ARM64 calling convention.
- Demonstrate system call or libc interaction.
- Analyze runtime memory layout and process behavior.

2. Motivation

Operating Systems courses cover:

- Process execution and program loading
- Memory layout (text, data, stack, heap)
- Calling conventions
- System calls
- Stack growth and recursion

However, students rarely observe how:

- High-level source code transforms into machine instructions
- Stack frames are constructed
- ABI rules are enforced at the assembly level
- Executable object binaries are structured
- Syscalls transition execution into kernel mode

Broccoli aims to bridge this gap by implementing a compiler that produces ARM64 assembly whose runtime behavior can be analyzed using OS-level tools.

3. Project Objectives

3.1 Primary Objectives

1. Implement a working C subset compiler in Kotlin.
2. Generate ARM64 assembly for Apple Silicon.
3. Produce valid Mach-O executables via `clang` or `ld`.
4. Implement correct ARM64 function prologue and epilogue.
5. Demonstrate recursion and stack growth.
6. Analyze runtime memory layout.

3.2 Secondary Objectives

- Implement constant folding.
- Compare generated assembly with Clang.
- Measure stack usage and instruction count differences.

4. Language Subset

4.1 Supported Features

- Data types: `int`, `char`
- Local and global variables
- Unary Operators
- Arithmetic Operators
- Relational and Logical operators
- Bit-wise Operators
- `if-else`
- `while` and `for`
- Functions with parameters
- Recursion
- Return statements

4.2 Non-Goals

- Full C standard compliance
- Preprocessor
- Structs/unions
- Dynamic memory allocation
- Multi-file compilation
- Advanced optimization passes
- Detailed formal documentation beyond implementation report
- TAC as required evaluation artifact (used only internally if needed)

5. Technical Architecture

5.1 Compiler Pipeline

1. Lexical Analysis
2. Recursive Descent Parsing
3. AST Construction
4. Semantic Analysis (symbol table + type checking)
5. ARM64 Assembly Code Generation

5.2 ARM64 Code Generation Strategy

Target platform: Apple Silicon (AArch64).

Key considerations:

- Argument registers: `x0--x7`
- Return value: `x0`
- Frame pointer: `x29`
- Link register: `x30`
- Stack alignment: 16 bytes
- Function prologue:

```
stp x29, x30, [sp, -16]!  
mov x29, sp
```

- Function epilogue:

```
ldp x29, x30, [sp], 16  
ret
```

6. Detailed Timeline (8 Weeks)

Week 0-1

- Define grammar and features for C subset
- Kotlin familiarisation and study

Week 1-2

- Define basic tokens such as identifiers, operators, punctuation, keywords, etc.
- Implement lexer for these tokens
- Construct AST classes

Week 2-3

- Add local variable support and compound assignments
- Recalibrate the lexer
- Assembly generation and testing

Week 3-4

- Implement control-flow and conditionals
- Add support for compound statements
- Implement constant folding

Week 4-5

- Implement for and do-while loops
- Scope Management

Week 5-6

- Function support and creation implementation
- Add Recursion features
- Measure stack usage

Week 6-7

- Global variable scope and support
- Added flexibility in variable usage
- Implement additional features if time permits

Week 7-8

- Tying in all the features
- Kernel-level syscall experiments
- Final debugging
- Prepare evaluation demo

7. Deliverables

- Kotlin source code of Broccoli
- ARM64 assembly outputs
- Sample compiled programs
- Mach-O executables
- Comparative analysis (Broccoli vs Clang)
- Demonstration of runtime stack behavior

8. Evaluation Criteria

1. Correct parsing and semantic validation
2. Valid ARM64 assembly generation
3. Proper stack frame construction
4. Successful Mach-O executable generation
5. Demonstrated understanding of OS-level execution